

Stegstr Contest Entry

Robust Steganographic Networking — Surviving WhatsApp, Instagram & Telegram Recompression

Submitted by: Muhammad Akif Janjua

GitHub: github.com/akifjanjua/Stegstr

Base project: brunkstr/Stegstr (forked and improved)

Headline result

20 out of 20 pass — every platform tested (WhatsApp, Instagram, Telegram, Facebook, Twitter) × every cover-image type tested — in a channel simulator that precisely replicates each platform's real resize + JPEG re-compression pipeline. Confirmed again with real WhatsApp and Instagram sends (see “Validation — real platform sends” below), and again after expanding the test set to 9 realistic cover types including a real phone-camera aspect ratio and a screenshot-style image (45/45, see “Deeper validation” below). The reference implementation's own documentation states it does **not** survive this — this entry fixes that, which is this contest's core, judged requirement.

The problem

The reference Stegstr app embeds hidden data in the spatial-domain pixel LSBs of a PNG (a DWT/Haar wavelet scheme). Every platform this contest requires surviving — WhatsApp, Telegram, Instagram — re-encodes uploaded photos as JPEG. JPEG re-compression recomputes every pixel from scratch, which destroys spatial-domain LSBs completely. This isn't a hidden bug: the project's own wiki says so directly.

“Format: PNG (lossless). Avoid JPEG or other lossy formats — recompression can destroy the hidden data.” — Stegstr Wiki, How It Works

In other words: the reference app works only if nobody actually shares the image through the platforms this contest is testing against.

The fix

Embedding was moved into the JPEG DCT coefficient domain itself — the exact domain every one of these platforms' own re-compression operates in — using Quantization Index Modulation (QIM), with three specific design decisions that made the difference between failing and passing:

- **Universal safe pre-resize, never a platform guess.** Coefficient-domain embedding cannot survive an actual pixel resize — resampling recomputes every 8×8 block from scratch (confirmed at ~50% bit-error rate, i.e. random data). Rather than guessing which single platform an image will be sent through and pre-matching that width, embedding now always shrinks the cover to a width safe for every platform in the target set, so every downstream resize becomes a guaranteed no-op.
- **Spatially decorrelated redundancy.** The original 5× bit-repetition placed every redundant copy of a bit in the same 8×8 block, so one bad block (e.g. a flat, low-detail region) made every copy wrong the

same way — majority voting over correlated failures does nothing. A single fixed pseudorandom permutation of every coefficient position (identical on encode and decode) now scatters redundant copies across genuinely distant parts of the image.

- **Reed–Solomon error correction with confidence-based erasures**, with a fallback to plain blind correction when erasure-assisted decode fails — the confidence heuristic can over-trigger on flat, low-detail content (see “Deeper validation” below) even when the underlying signal is still fine.

Validation — simulated channel matrix

Tested against a channel simulator that replicates each platform's actual pipeline (strip metadata, resize to the platform's real max width, re-encode as JPEG at the platform's real quality setting), using four realistic (non-flat) cover images — a textured photo, a portrait-like image, a high-frequency noise image, and a smooth gradient (a deliberately adversarial worst case with almost no detail to hide data in).

Cover image	WhatsApp	Instagram	Telegram	Facebook	Twitter/X
Textured photo	PASS	PASS	PASS	PASS	PASS
Portrait-like	PASS	PASS	PASS	PASS	PASS
High-frequency noise	PASS	PASS	PASS	PASS	PASS
Smooth gradient (worst case)	PASS	PASS	PASS	PASS	PASS

20/20 pass. Full methodology, raw bit-error-rate measurements, and the tuning process are documented in `channel_simulator/BASELINE_RESULTS.md` in the repository, reproducible with `python run_matrix_realistic.py`.

Validation — real platform sends

Simulation is a proxy; the contest is judged on real platforms, so the same build was tested against them directly. A JPEG photo was embedded with the hidden message “This is a hidden test message from Stegstr!”, sent as a normal photo (not a file/document) through WhatsApp and Instagram, downloaded from the receiving side, and decoded with the built CLI.

Platform	File size change	Decoded message
WhatsApp (desktop)	No change (89,697 bytes)	“This is a hidden test message from Stegstr!”
Instagram (DM)	Recompressed (89,697 → 97,513 bytes)	“This is a hidden test message from Stegstr!”

Message recovered exactly, byte-for-byte, on both platforms — including the Instagram case, where the platform genuinely re-encoded the file (confirmed by the file size actually changing), which is the harder and more realistic test of the two. Telegram was validated only via the simulator above (no Telegram account available for a live test); it uses identical code to the two platforms confirmed live.

Deeper validation: 9 realistic cover types, real bugs found and fixed

The initial 4-cover validation above used square 768×768 synthetic images. To push harder before calling this finished, the cover set was expanded to 9 types — including a real phone-camera aspect ratio (1080×1920 portrait, since no phone camera actually shoots square), a screenshot/UI-style image, low-light, high-contrast, and a deliberately adversarial narrow-but-tall case. This surfaced two real bugs, both fixed:

- The pre-resize safety check only looked at image *width*; real platforms constrain by the *longer* side. A narrow-but-tall cover could pass unshrunk and then get resized for real downstream — exactly the failure mode the pre-resize exists to prevent.
- The Reed–Solomon confidence-based erasure marking over-triggered on flat, low-detail content (screenshots), flagging more bytes as unrecoverable than the codeword could mathematically correct — even though the underlying signal was still fine (~1% raw bit-error rate). Fixed by falling back to plain blind error correction when erasure-assisted decode fails.

Result after both fixes, run through the actual compiled Rust CLI (not just the research prototype): **45/45** — 9 cover types × all 5 platforms.

Invisibility, measured (not just assumed)

Steganographic invisibility is one of this contest's three core judged criteria, alongside robustness and networking reliability — so it was measured directly rather than left as an assumption. All earlier tuning optimized only bit-error-rate; checking the actual visual output found the original QIM_DELTA=32 setting scored just ~26dB PSNR against a same-pipeline, no-embedding baseline (identical resize and JPEG quality, zero steganographic changes) — genuinely visible graininess on direct inspection, confirmed by eye as well as by the number, not just a technicality.

Setting	Robustness (9 covers × 5 platforms)	Image quality (PSNR vs. clean baseline)
Original (QIM_DELTA=32)	45/45	~26 dB — visibly grainy
Re-tuned (QIM_DELTA=16)	45/45	~32 dB — clean

Re-swept the quantization step with PSNR measured alongside robustness at every value tested, not bit-error-rate alone. QIM_DELTA=16 keeps the identical full robustness result while roughly halving the visible error — a real quality improvement with no correctness cost, made possible because the redundancy and error-correction fixes above meant far less raw signal margin was actually needed than the original bit-error-only tuning assumed.

Also audited: two more implementations, one more bug found

This codebase turned out to contain *three* independent implementations of the same embedding scheme: the CLI/desktop app (Rust), a research prototype (Python), and a separate browser/web build used by `stegstr.com/app` (TypeScript). The web implementation had never been tested against real platform compression at all, and had the exact same confirmed-bad pattern the desktop app originally had: pre-resizing to a *guessed* destination platform instead of a universal safe width. Fixed the same way as the Rust/Python side. Also verified: a realistic 5-recipient encrypted payload round-trips correctly through all 5 platforms, and decode no longer prints an alarming-looking message when handed an ordinary non-Stegstr image (a routine case, not a bug, but was being routed through a path that looked like a crash in logs even though it wasn't one).

Beyond the core requirement

Networking reliability. Two real gaps found and fixed in the Nostr relay client:

- Dropped relay connections never reconnected (a brief WiFi blip or relay restart silently killed sync until the app was restarted). Added reconnection with exponential backoff.
- Publishing a post or DM was fire-and-forget with zero confirmation — a user could believe a message sent when it silently reached no relay at all. Publishing now tracks relay OK confirmations and surfaces a failure to the user instead of failing silently.

AI agent operability. The CLI's `embed/decode/detect` commands now support the robust encoder (`--robust / --robustness standard|max`) and transparently try both the robust and original decoders, so an agent or script decoding an unknown image doesn't need to know in advance which encoder produced it.

How to build and verify

From the repository root:

```
cd src-tauri
cargo build --release --bin stegstr-cli

target\release\stegstr-cli.exe embed cover.jpg -o out.jpg --robust --payload "hello world"
target\release\stegstr-cli.exe decode out.jpg
```

Full setup notes, what was changed and why, and verification status (including an honest note on what wasn't able to be fully browser-tested) are in `ROBUSTNESS_PORT_NOTES.md` and `channel_simulator/BASELINE_RESULTS.md` in the repository.

Submission

Full source: <https://github.com/akifjanjua/Stegstr>

MIT licensed, forked from and built on top of the original `brunkstr/Stegstr` project.